

Technical Assessment - Take-Home Task

Role: Backend / Automation Engineer (Python) · Projectx

Suggested effort: 3–5 hours

Deadline: 2 July 2026

Submit to : ibrahim@projectx.io
info@projectx.io

Objective

Build a small **Flask API** that, when called, triggers an automated workflow: it searches Google for YouTube mobile-review videos, opens the top results in the **YouTube mobile app**, extracts each video's details, and returns them as a structured JSON response.

This task mirrors the real work of the role — combining backend/API development, browser automation, and mobile app automation into one reliable, unattended workflow.

The Workflow

When your API endpoint is called, the service should:

1. **Trigger via API** — expose a Flask endpoint (e.g. `GET /reviews`) that starts the workflow.
2. **Google search (via browser)** — open a browser and search Google for: “**Mobile review youtube**”.
3. **Take the first 3 results** — identify the first three YouTube video results on the search page.
4. **Open each in the YouTube app** — open each result via its link so it launches in the **installed YouTube mobile app** (deep-link into the app, not the web page).
5. **Extract video details** — from each opened video read the **author (channel name)**, **description**, and **video URL**.
6. **Return the data** — respond from the Flask API with all three results as JSON.

Expected Response (example shape)

```
{
  "query": "Mobile review youtube",
  "results": [
    {
      "author": "Channel Name",
      "description": "Full review of the latest ...",
      "video_url": "https://www.youtube.com/watch?v=..."
    },
    { "author": "...", "description": "...", "video_url": "..." },
    { "author": "...", "description": "...", "video_url": "..." }
  ]
}
```

Technical Requirements

- **Language:** Python.
- **API:** Flask — a single GET endpoint is enough.
- **Browser automation** for the Google search step (Selenium, Playwright, or Appium — your choice).

- **Mobile automation** to open and read from the YouTube app (Appium or equivalent; an Android emulator or a real device is fine).
- **Use explicit waits** — no blind `sleep()` calls.
- **Handle failures gracefully** — if a result fails to load, return a clear error rather than crashing.
- **No hardcoded secrets** — load any credentials/config from environment variables.
- **README** — explain your setup, the tools you chose, how to run it, and any trade-offs or assumptions.

Deliverables

- A **GitHub repository link** (preferred) or a **zip file** with your code and README.
- A short note — or an optional 1–2 minute screen recording — showing it run end-to-end.

How We Evaluate

Criterion	What we look for
Correctness	Workflow runs end-to-end and returns the 3 videos with author, description, and URL.
Reliability	Explicit waits, graceful error handling, no flaky behaviour on re-runs.
Automation craft	Stable locators and a sound approach to the browser → app hand-off.
Code quality	Clean structure, readable code, sensible naming, clear README.
Bonus	Idempotency, logging, screenshot-on-failure, retry/backoff, tests, or Dockerised setup.

A few notes: You may use an Android emulator or a physical device — both are fine. If Google search proves unreliable to automate (e.g. consent screens), document how you handled it. We care far more about how you reason about reliability and structure your code than about pixel-perfect polish. If you make an assumption, note it in your README.

Submission

Send your GitHub link or zip file to ibrahim@projecx.io or info@projecx.io by **2 July 2026**. If anything is unclear, reply to the assessment email and we'll be glad to help. Good luck — we're looking forward to seeing your approach.